

solution and prints the solved puzzle in the matrix grids.

`initArray()` function initialises the array of numbers from 1 to 81 that has numbers in range of 1 to 9. It also checks whether the provided input is a number or not. And if it is not, it initialises the first grid value to 1.

`check()` function handles the position of the grid and number, and arranges the same accordingly to the rule that no two numbers should appear in the same row or same column.

`Math.floor()` function from JS Math library returns the value which are less than that is provided, i.e., if 9 is

provided as a parameter, it returns numbers from 1 to 8 and not 9.

`Math.ceil()` function returns a value which is higher than the input parameter.

EFY note. The source code of this article has been included in this month's EFY-CD.

SENSOR BASED PATH FINDER PROGRAM IN C



■ MANUJ ARORA

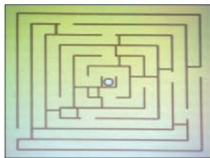
Micromouse is basically a robotics-based path finder application that consists of a micromouse and a maze having complex tracks. The micromouse is so designed that it can move in dark to reach the centre of the maze using signals received from its sensors. The concept of this program can be used to develop an autonomous machine (spy machine) that can explore the enemy premises and gather useful information.

Software program

The development of this project has two phases—designing and implementation. The maze is designed

using various concepts of C graphic functions and colours. The next step involves actual implementation of the micromouse—movements and activation of its sensors.

When the program is run, a blue micromouse, which had been at rest at one of the pre-defined entries, starts



Screenshot: Micromouse at the centre of the maze

moving. It heads straight in forward path until it reaches an opening on its left or right side. When it encounters such an opening, it stops for a second and changes colour from blue to red. The sensors get activated at this point and enable the micromouse to take a 90 degree turn towards a selected path. It then continues to move straight along the path and reaches the final destination, i.e., the centre of the maze.

The output of the program is shown in the screenshot. Before running the program make sure that `egavga`, `tscr.chr` and `scri.chr` graphic files are in the directory where executable file is located.

EFY note. The source code of this article has been included in this month's EFY-CD. ●

ELECTRONICS

BAZAAR

INDIA'S FIRST ELECTRONICS SOURCING MAGAZINE

COMPONENTS. PRODUCTS. MACHINERY.

